

```
=====
PROPLET - EVIDENCE VAULT IMPLEMENTATION PLAN
=====
```

```
Scope: Per-tenancy PDF evidence bundle generation
Tech: Puppeteer (headless Chrome), AWS S3, React Router v7
One click on the tenancy detail page generates a complete
PDF evidence bundle for that tenancy.
=====
```

```
=====
SECTION 1 - WHAT THE EVIDENCE VAULT IS
=====
```

The evidence vault is a one-click PDF generator that takes everything Proplet knows about a tenancy and produces a single, professionally formatted PDF document.

This PDF is what an agency hands to:

- A council inspector asking for a compliance file
- A solicitor pursuing Section 8 possession
- A Property Ombudsman adjudicator in a dispute
- A landlord asking for a tenancy summary

The bundle is generated on demand. It is not stored permanently -- it is generated fresh each time so it always reflects the current state of the tenancy record.

WHAT THE PDF CONTAINS:

1. Cover page -- tenancy summary and generation date
2. Property details -- address, type, certificates
3. Landlord details -- name, AML status
4. Tenant details -- name, RTR status (per tenant)
5. Tenancy details -- type, dates, rent, deposit
6. Compliance checklist -- all obligations and status
7. Certificate history -- gas, EICR, EPC records
8. Deposit tracking -- protection details, prescribed info
9. How to Rent -- served date and version
10. Section 8 notices -- all notices with grounds and outcome
11. Section 13 notices -- all rent increase notices
12. Rent payment history -- 12 months of payments
13. Document index -- list of all uploaded documents
14. Audit trail -- key actions with timestamps and who did them

LENGTH: typically 8-15 pages depending on tenancy history

```
=====
SECTION 2 - ARCHITECTURE
=====
```

APPROACH: Server-side PDF generation using Puppeteer

- Agent clicks Generate Evidence Bundle on tenancy detail
- POST request sent to server action
- Server fetches all tenancy data from MongoDB
- Server renders an HTML template with that data
- Puppeteer converts HTML to PDF
- PDF uploaded to S3 with a unique key
- Presigned download URL returned to browser
- Browser triggers file download automatically

WHY PUPPETEER NOT PDFKIT:

- Puppeteer renders HTML/CSS to PDF.
- You write the bundle as an HTML template with Tailwind.
- No learning a separate PDF API.
- Full control over layout, tables, and styling.
- Produces professional output that matches your brand.

WHY NOT AWS LAMBDA:

- Lambda adds complexity for a feature that does not need to scale massively. A Vercel serverless function with Puppeteer is simpler and sufficient at your scale.
- Revisit Lambda if generation takes too long at scale.

FILE STRUCTURE:

```
app/
  routes/
    api.evidence-vault.$tenancyId.generate.tsx
    -- POST: generates the PDF, returns download URL

  lib/
    evidenceVault.server.ts
    -- fetchBundleData(tenancyId, agencyId)
    -- generatePDF(bundleData)
    -- uploadPDFToS3(pdfBuffer, tenancyId, agencyId)

  templates/
    evidence-bundle.html
    -- standalone HTML template with inline CSS
    -- Puppeteer renders this to PDF
```

```
components/  
  EvidenceVaultButton.tsx  
  -- the Generate Bundle button on tenancy detail page  
  -- handles loading state and download trigger
```

```
=====
```

SECTION 3 - DATA FETCHING

```
=====
```

```
// lib/evidenceVault.server.ts
```

```
export async function fetchBundleData(tenancyId, agencyId) {
```

```
  // — 1. Tenancy —————
```

```
  const tenancy = await Tenancy.findOne({  
    _id: tenancyId,  
    agencyId,  
    deleted: false,  
  })  
    .populate('propertyId')  
    .populate('landlordId', 'firstName lastName email phone landlordData')  
    .populate('tenantIds', 'firstName lastName email phone tenantData address')  
    .populate('createdBy', 'firstName lastName')  
    .lean();
```

```
  if (!tenancy) throw new Error('Tenancy not found');
```

```
  // — 2. All documents for this tenancy —————
```

```
  const documents = await Document.find({  
    entityType: 'tenancy',  
    entityId: tenancyId,  
    agencyId,  
  })  
    .populate('uploadedBy', 'firstName lastName')  
    .populate('verifiedBy', 'firstName lastName')  
    .sort({ createdAt: -1 })  
    .lean();
```

```
  // — 3. Property certificates —————
```

```
  const propertyDocuments = await Document.find({  
    entityType: 'property',  
    entityId: tenancy.propertyId._id,  
    agencyId,  
    docType: { $in: ['gas_certificate', 'eicr_report', 'epc_certificate'] },  
  })
```

```

        .sort({ createdAt: -1 })
        .lean();

// Most recent certificate of each type
const certificates = {
  gas: propertyDocuments.find(d => d.docType === 'gas_certificate') || null,
  eicr: propertyDocuments.find(d => d.docType === 'eicr_report') || null,
  epc: propertyDocuments.find(d => d.docType === 'epc_certificate') || null,
};

// — 4. Rent payments —————
const rentPayments = await RentPayment.find({
  tenancyId,
  agencyId,
  deleted: false,
})
  .populate('recordedBy', 'firstName lastName')
  .sort({ periodStart: -1 })
  .limit(24) // last 24 months max
  .lean();

// — 5. Arrears status —————
const arrearsStatus = calculateArrearsStatus(rentPayments);

// — 6. Compliance status —————
const complianceStrip = buildComplianceStrip(
  tenancy.propertyId,
  certificates
);

// — 7. Agency details —————
const agency = await Agency.findById(agencyId).lean();

// — 8. Generation metadata —————
const generatedAt = new Date();
const generatedBy = null; // passed from action separately

return {
  tenancy,
  documents,
  certificates,
  rentPayments,
  arrearsStatus,
  complianceStrip,
  agency,
  generatedAt,
};

```

```
}
```

```
=====
SECTION 4 - PDF GENERATION
=====
```

```
// lib/evidenceVault.server.ts (continued)

import puppeteer from 'puppeteer';
import { renderBundleHTML } from './bundleTemplate.server';

export async function generatePDF(bundleData) {
  // Render HTML from template
  const html = renderBundleHTML(bundleData);

  // Launch Puppeteer
  const browser = await puppeteer.launch({
    headless: true,
    args: [
      '--no-sandbox',
      '--disable-setuid-sandbox',
      '--disable-dev-shm-usage',
      // required for Vercel/serverless environment
    ],
  });

  const page = await browser.newPage();

  // Set content and wait for all resources to load
  await page.setContent(html, { waitUntil: 'networkidle0' });

  // Generate PDF with professional settings
  const pdfBuffer = await page.pdf({
    format: 'A4',
    printBackground: true,
    margin: {
      top: '20mm',
      right: '15mm',
      bottom: '20mm',
      left: '15mm',
    },
    displayHeaderFooter: true,
    headerTemplate: `
    <div style="font-size:9px; width:100%; padding:0 15mm;
      display:flex; justify-content:space-between;
      color:#64748B; font-family:Arial;">
```

```

        <span>PROPLET - EVIDENCE BUNDLE</span>
        <span>CONFIDENTIAL</span>
    </div>
    `,
    footerTemplate: `
    <div style="font-size:9px; width:100%; padding:0 15mm;
    display:flex; justify-content:space-between;
    color:#64748B; font-family:Arial;">
    <span>Generated: ${new Date().toLocaleDateString('en-GB')}</span>
    <span>Page <span class="pageNumber"></span> of
        <span class="totalPages"></span></span></span>
    </div>
    `,
    });

    await browser.close();
    return pdfBuffer;
}

```

```

export async function uploadBundleToS3(pdfBuffer, tenancyId, agencyId) {
    // S3 key for this bundle
    // Overwritten each time -- not versioned
    // Agent always gets the freshest bundle
    const s3Key = `agency/${agencyId}/evidence-bundles/${tenancyId}/bundle.pdf`;

    await uploadToS3(s3Key, pdfBuffer, 'application/pdf');

    // Generate presigned download URL (1 hour expiry)
    const downloadUrl = await getPresignedUrl(s3Key, 3600);

    return { s3Key, downloadUrl };
}

```

```

=====
SECTION 5 - SERVER ACTION
=====

```

```

// routes/api.evidence-vault.${tenancyId}.generate.tsx

export async function action({ request, params }) {
    const user = await requireAuth(request);

    const tenancyId = params.tenancyId;

    // — 1. Verify tenancy belongs to this agency —————

```

```

const tenancy = await Tenancy.findOne({
  _id: tenancyId,
  agencyId: user.agencyId,
  deleted: false,
}).lean();

if (!tenancy) {
  throw new Response('Not Found', { status: 404 });
}

// — 2. Fetch all bundle data —————
const bundleData = await fetchBundleData(tenancyId, user.agencyId);
bundleData.generatedBy = `${user.firstName} ${user.lastName}`;

// — 3. Generate PDF —————
let pdfBuffer;
try {
  pdfBuffer = await generatePDF(bundleData);
} catch (error) {
  console.error('PDF generation failed:', error);
  return { error: 'Failed to generate PDF. Please try again.' };
}

// — 4. Upload to S3 —————
const { downloadUrl } = await uploadBundleToS3(
  pdfBuffer,
  tenancyId,
  user.agencyId
);

// — 5. Log bundle generation in audit trail —————
await Note.create({
  agencyId: user.agencyId,
  entityType: 'tenancy',
  entityId: tenancyId,
  text: `Evidence bundle generated by ${user.firstName}
${user.lastName}.`,
  addedBy: user._id,
});

// — 6. Return download URL —————
return { success: true, downloadUrl };
}

```

=====

The bundle is an HTML file that Puppeteer renders.
Write it as a single self-contained HTML file with
inline styles (no external CSS -- Puppeteer cannot
load external stylesheets reliably in serverless).

```
// lib/bundleTemplate.server.ts
```

```
export function renderBundleHTML(data) {
  const { tenancy, documents, certificates,
    rentPayments, arrearsStatus,
    complianceStrip, agency, generatedAt,
    generatedBy } = data;

  return `
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; font-size: 11px;
      color: #1E293B; line-height: 1.5; }
    h1 { font-size: 20px; color: #1E293B; margin-bottom: 8px; }
    h2 { font-size: 14px; color: #2563EB; margin: 20px 0 8px;
      padding-bottom: 4px; border-bottom: 2px solid #DBEAFE; }
    h3 { font-size: 12px; color: #1E293B; margin: 12px 0 6px; }
    .cover { background: #2563EB; color: white; padding: 40px;
      margin: -20mm -15mm 0; }
    .cover h1 { font-size: 28px; color: white; }
    .cover p { color: #BFDBFE; font-size: 13px; margin-top: 8px; }
    .meta { display: flex; gap: 40px; margin-top: 20px; }
    .meta-item { color: #93C5FD; font-size: 11px; }
    .meta-item strong { color: white; display: block;
      font-size: 13px; }
    .section { margin-bottom: 20px; }
    .grid-2 { display: grid; grid-template-columns: 1fr 1fr;
      gap: 12px; }
    .card { background: #EFF6FF; padding: 12px;
      border-radius: 4px; }
    .card label { font-size: 10px; color: #64748B;
      display: block; margin-bottom: 2px; }
    .card value { font-size: 11px; color: #1E293B; }
    table { width: 100%; border-collapse: collapse;
      margin-top: 8px; }
    th { background: #2563EB; color: white; padding: 6px 8px;
```

```

        text-align: left; font-size: 10px; }
td { padding: 5px 8px; font-size: 10px; color: #1E293B;
    border-bottom: 1px solid #E2E8F0; }
tr:nth-child(even) td { background: #EFF6FF; }
.badge { display: inline-block; padding: 2px 8px;
    border-radius: 3px; font-size: 9px;
    font-weight: bold; }
.badge-green { background: #DCFCE7; color: #16A34A; }
.badge-amber { background: #FEF3C7; color: #D97706; }
.badge-red { background: #FEE2E2; color: #DC2626; }
.badge-grey { background: #F1F5F9; color: #64748B; }
.badge-blue { background: #DBEAFE; color: #2563EB; }
.check-item { display: flex; gap: 8px; padding: 6px 0;
    border-bottom: 1px solid #F1F5F9; }
.check-icon { width: 16px; font-size: 12px; }
.check-label { flex: 1; }
.check-date { color: #64748B; font-size: 10px; }
.disclaimer { margin-top: 30px; padding: 12px;
    background: #FEF3C7; border-radius: 4px;
    font-size: 9px; color: #92400E; }
.page-break { page-break-after: always; }
</style>
</head>
<body>

<!-- COVER PAGE -->
<div class="cover">
    <div style="font-size:11px; color:#93C5FD; margin-bottom:8px;">
        EVIDENCE BUNDLE
    </div>
    <h1>${tenancy.propertyId.addressLine1}</h1>
    <p>${tenancy.propertyId.city}, ${tenancy.propertyId.postcode}</p>
    <div class="meta">
        <div class="meta-item">
            <strong>${formatDate(tenancy.startDate)} -
                ${tenancy.endDate}
                ? formatDate(tenancy.endDate)
                : 'Periodic (APT)'}</strong>
            Tenancy Period
        </div>
        <div class="meta-item">
            <strong>£${tenancy.rent.amount.toLocaleString()} pcm</strong>
            Monthly Rent
        </div>
        <div class="meta-item">
            <strong>${tenancy.tenancyType.toUpperCase()}</strong>
            Tenancy Type

```

```

    </div>
    <div class="meta-item">
        <strong>${formatDate(generatedAt)}</strong>
        Generated
    </div>
</div>
<div style="margin-top:20px; font-size:10px; color:#93C5FD;">
    Generated by ${generatedBy} | ${agency.name}
</div>
</div>

<div class="page-break"></div>

<!-- SECTION 1: PROPERTY DETAILS -->
<div class="section">
    <h2>1. Property Details</h2>
    <div class="grid-2">
        <div class="card">
            <label>Address</label>
            <div>${tenancy.propertyId.addressLine1}
                ${tenancy.propertyId.addressLine2 || ''}
                ${tenancy.propertyId.city}
                ${tenancy.propertyId.postcode}
            </div>
        </div>
        <div class="card">
            <label>Property Type</label>
            <div>${formatPropertyType(tenancy.propertyId.propertyType)}</div>
            <label style="margin-top:8px;">Local Authority</label>
            <div>${tenancy.propertyId.localAuthority || 'Not recorded'}</div>
        </div>
        <div class="card">
            <label>EPC Rating</label>
            <div>${tenancy.propertyId.epcRating || 'Not recorded'}
                ${tenancy.propertyId.epcExpiryDate}
                ? '(Expires: ' + formatDate(tenancy.propertyId.epcExpiryDate) + ')'
                : ''</div>
        </div>
        <div class="card">
            <label>HMO</label>
            <div>${tenancy.propertyId.propertyType === 'hmo'
                ? 'Yes - Licence: ' + (tenancy.propertyId.hmoLicenceNo || 'Not
recorded')
                : 'Not an HMO'}</div>
        </div>
    </div>
</div>
</div>

```

```

<!-- SECTION 2: LANDLORD DETAILS -->
<div class="section">
  <h2>2. Landlord Details</h2>
  <div class="grid-2">
    <div class="card">
      <label>Name</label>
      <div>${tenancy.landlordId.firstName} ${tenancy.landlordId.lastName}</div>
      <label style="margin-top:8px;">Email</label>
      <div>${tenancy.landlordId.email}</div>
      <label style="margin-top:8px;">Phone</label>
      <div>${tenancy.landlordId.phone || 'Not recorded'}</div>
    </div>
    <div class="card">
      <label>AML Check</label>
      <div>
        <span class="badge ${getAmlBadgeClass(
          tenancy.landlordId.landlordData?.amlResult
        )}">
          ${tenancy.landlordId.landlordData?.amlResult?.toUpperCase()
            || 'PENDING'}
        </span>
      </div>
      <label style="margin-top:8px;">AML Check Date</label>
      <div>${tenancy.landlordId.landlordData?.amlCheckedAt
        ? formatDate(tenancy.landlordId.landlordData.amlCheckedAt)
        : 'Not completed'}</div>
      <label style="margin-top:8px;">PEP Check</label>
      <div>${tenancy.landlordId.landlordData?.pepResult
        ? tenancy.landlordId.landlordData.pepResult.toUpperCase()
        : 'Not completed'}</div>
    </div>
  </div>
</div>

<!-- SECTION 3: TENANT DETAILS -->
<div class="section">
  <h2>3. Tenant Details</h2>
  ${tenancy.tenantIds.map(tenant => `
    <h3>${tenant.firstName} ${tenant.lastName}</h3>
    <div class="grid-2">
      <div class="card">
        <label>Email</label><div>${tenant.email}</div>
        <label style="margin-top:8px;">Phone</label>
        <div>${tenant.phone || 'Not recorded'}</div>
        <label style="margin-top:8px;">Address</label>
        <div>${formatAddress(tenant.address)}</div>

```

```

</div>
<div class="card">
  <label>Right to Rent</label>
  <div>
    <span class="badge ${getRTRBadgeClass(tenant)}">
      ${getRTRLabel(tenant)}
    </span>
  </div>
  <label style="margin-top:8px;">Document Type</label>
  <div>${formatRTRDocType(tenant.tenantData?.rightToRentDocumentType)}
</div>

  <label style="margin-top:8px;">Check Date</label>
  <div>${tenant.tenantData?.rightToRentCheckDate
    ? formatDate(tenant.tenantData.rightToRentCheckDate)
    : 'Not recorded'}</div>
  <label style="margin-top:8px;">Expiry</label>
  <div>${tenant.tenantData?.rightToRentExpiry
    ? formatDate(tenant.tenantData.rightToRentExpiry)
    : 'No expiry (UK/Irish citizen)'}</div>
</div>
</div>
`.join(''))}
</div>

```

```

<!-- SECTION 4: TENANCY DETAILS -->

```

```

<div class="section">
  <h2>4. Tenancy Details</h2>
  <div class="grid-2">
    <div class="card">
      <label>Tenancy Type</label>
      <div>${tenancy.tenancyType === 'apt'
        ? 'Assured Periodic Tenancy (APT)'
        : 'Assured Shorthold Tenancy (AST)'}</div>
      <label style="margin-top:8px;">Start Date</label>
      <div>${formatDate(tenancy.startDate)}</div>
      <label style="margin-top:8px;">End Date</label>
      <div>${tenancy.endDate
        ? formatDate(tenancy.endDate)
        : 'Periodic - no fixed end date'}</div>
      <label style="margin-top:8px;">Status</label>
      <div><span class="badge badge-${tenancy.status === 'active'
        ? 'green' : 'grey'}">${tenancy.status.toUpperCase()}</span>
      </div>
    </div>
    <div class="card">
      <label>Monthly Rent</label>
      <div>£${tenancy.rent.amount.toLocaleString()}</div>
    </div>
  </div>

```

```

<label style="margin-top:8px;">Rent Due Day</label>
<div>${tenancy.rent.dueDay}${ordinal(tenancy.rent.dueDay)} of month</div>
<label style="margin-top:8px;">Payment Method</label>
<div>${formatPaymentMethod(tenancy.rent.paymentMethod)}</div>
<label style="margin-top:8px;">Next Rent Review</label>
<div>${tenancy.rent.reviewDate
    ? formatDate(tenancy.rent.reviewDate)
    : 'Not set'}</div>
</div>
</div>
</div>

```

```

<!-- SECTION 5: DEPOSIT -->

```

```

<div class="section">
  <h2>5. Deposit Protection</h2>
  <div class="grid-2">
    <div class="card">
      <label>Deposit Amount</label>
      <div>${tenancy.deposit.amount
        ? '£' + tenancy.deposit.amount.toLocaleString()
        : 'No deposit taken'}</div>
      <label style="margin-top:8px;">Protection Scheme</label>
      <div>${tenancy.deposit.scheme
        ? tenancy.deposit.scheme.toUpperCase()
        : 'Not protected'}</div>
      <label style="margin-top:8px;">Reference</label>
      <div>${tenancy.deposit.reference || 'Not recorded'}</div>
    </div>
    <div class="card">
      <label>Protected Date</label>
      <div>${tenancy.deposit.protectedDate
        ? formatDate(tenancy.deposit.protectedDate)
        : getDepositStatus(tenancy)}</div>
      <label style="margin-top:8px;">Prescribed Info Served</label>
      <div>${tenancy.deposit.prescribedInfoServedDate
        ? formatDate(tenancy.deposit.prescribedInfoServedDate)
        : getPrescrInfoStatus(tenancy)}</div>
    </div>
  </div>
</div>
</div>

```

```

<!-- SECTION 6: HOW TO RENT -->

```

```

<div class="section">
  <h2>6. How to Rent Guide</h2>
  <div class="grid-2">
    <div class="card">
      <label>Served to Tenant</label>

```

```

<div>${tenancy.howToRent.served
  ? 'Yes'
  : '<span class="badge badge-red">NOT SERVED</span>'}</div>
<label style="margin-top:8px;">Date Served</label>
<div>${tenancy.howToRent.servedDate
  ? formatDate(tenancy.howToRent.servedDate)
  : 'Not recorded'}</div>
<label style="margin-top:8px;">Version Served</label>
<div>${tenancy.howToRent.version || 'Not recorded'}</div>
</div>
</div>
</div>

```

```

<div class="page-break"></div>

```

```

<!-- SECTION 7: CERTIFICATES -->

```

```

<div class="section">
  <h2>7. Compliance Certificates</h2>
  <table>
    <tr>
      <th>Certificate</th>
      <th>Issue Date</th>
      <th>Expiry Date</th>
      <th>Status</th>
      <th>Uploaded By</th>
    </tr>
    <tr>
      <td><strong>Gas Safety Certificate</strong></td>
      <td>${certificates.gas
        ? formatDate(certificates.gas.createdAt) : '-'}</td>
      <td>${certificates.gas
        ? formatDate(certificates.gas.expiryDate) : '-'}</td>
      <td>${getCertBadge(certificates.gas)}</td>
      <td>${certificates.gas
        ? certificates.gas.uploadedBy?.firstName + ' ' +
          certificates.gas.uploadedBy?.lastName
        : 'Not uploaded'}</td>
    </tr>
    <tr>
      <td><strong>EICR</strong></td>
      <td>${certificates.eicr
        ? formatDate(certificates.eicr.createdAt) : '-'}</td>
      <td>${certificates.eicr
        ? formatDate(certificates.eicr.expiryDate) : '-'}</td>
      <td>${getCertBadge(certificates.eicr)}</td>
      <td>${certificates.eicr
        ? certificates.eicr.uploadedBy?.firstName + ' ' +

```

```

        certificates.eicr.uploadedBy?.lastName
        : 'Not uploaded'}</td>
</tr>
<tr>
    <td><strong>EPC</strong></td>
    <td>${certificates.epc
        ? formatDate(certificates.epc.createdAt) : '-'}</td>
    <td>${certificates.epc
        ? formatDate(certificates.epc.expiryDate) : '-'}</td>
    <td>${getCertBadge(certificates.epc)}</td>
    <td>${certificates.epc
        ? certificates.epc.uploadedBy?.firstName + ' ' +
          certificates.epc.uploadedBy?.lastName
        : 'Not uploaded'}</td>
</tr>
</table>
</div>

```

```

<!-- SECTION 8: SECTION 8 NOTICES -->

```

```

${tenancy.section8Notices.length > 0 ? `

```

```

<div class="section">

```

```

    <h2>8. Section 8 Notices</h2>

```

```

    <table>

```

```

        <tr>

```

```

            <th>Served Date</th>

```

```

            <th>Grounds</th>

```

```

            <th>Expiry Date</th>

```

```

            <th>Outcome</th>

```

```

            <th>Notes</th>

```

```

        </tr>

```

```

        ${tenancy.section8Notices.map(notice => `

```

```

            <tr>

```

```

                <td>${formatDate(notice.servedDate)}</td>

```

```

                <td>${notice.grounds.join(', ')}</td>

```

```

                <td>${formatDate(notice.expiryDate)}</td>

```

```

                <td><span class="badge badge-${getNoticeBadgeClass(notice.outcome)}">

```

```

                    ${notice.outcome?.toUpperCase() || 'PENDING'}

```

```

                </span></td>

```

```

                <td>${notice.notes || '-'}</td>

```

```

            </tr>

```

```

        `).join('')

```

```

    </table>

```

```

</div>

```

```

` : ''}

```

```

<!-- SECTION 9: SECTION 13 NOTICES -->

```

```

${tenancy.section13Notices.length > 0 ? `

```

```

<div class="section">
  <h2>9. Section 13 Rent Increase Notices</h2>
  <table>
    <tr>
      <th>Served Date</th>
      <th>Current Rent</th>
      <th>Proposed Rent</th>
      <th>Effective Date</th>
      <th>Outcome</th>
    </tr>
    ${tenancy.section13Notices.map(notice => `
      <tr>
        <td>${formatDate(notice.servedDate)}</td>
        <td>£${notice.currentRent.toLocaleString()}</td>
        <td>£${notice.proposedRent.toLocaleString()}</td>
        <td>${formatDate(notice.effectiveDate)}</td>
        <td><span class="badge badge-${getNoticeBadgeClass(notice.outcome)}">
          ${notice.outcome?.toUpperCase() || 'PENDING'}
        </span></td>
      </tr>
    `).join('')}
  </table>
</div>
` : ''}

```

```

<!-- SECTION 10: RENT PAYMENT HISTORY -->
<div class="section">
  <h2>10. Rent Payment History</h2>
  ${arrearsStatus.hasArrears ? `
    <div style="background:#FEE2E2; padding:8px 12px;
      border-radius:4px; margin-bottom:12px; font-size:10px;">
      <strong style="color:#DC2626;">Rent Arrears:</strong>
      £${arrearsStatus.totalArrears.toLocaleString()} outstanding
      (${arrearsStatus.monthsArrears} month(s) overdue)
      ${arrearsStatus.section8Ground8Eligible
        ? ' - Ground 8 Section 8 may be available.' : ''}
    </div>
  ` : ''}
  <table>
    <tr>
      <th>Period</th>
      <th>Due Date</th>
      <th>Amount Due</th>
      <th>Amount Paid</th>
      <th>Status</th>
      <th>Paid Date</th>
      <th>Method</th>
    </tr>
  </table>

```

```

</tr>
    ${rentPayments.map(p => `
        <tr>
            <td>${formatPeriod(p.periodStart)}</td>
            <td>${formatDate(p.dueDate)}</td>
            <td>£${p.amountDue.toLocaleString()}</td>
            <td>${p.amountPaid > 0
                ? '£' + p.amountPaid.toLocaleString() : '-'}</td>
            <td><span class="badge badge-${getPaymentBadgeClass(p.status)}">
                ${p.status.toUpperCase()}
            </span></td>
            <td>${p.paidDate ? formatDate(p.paidDate) : '-'}</td>
            <td>${p.paymentMethod
                ? formatPaymentMethod(p.paymentMethod) : '-'}</td>
        </tr>
    `).join('')}
</table>
</div>

<div class="page-break"></div>

<!-- SECTION 11: DOCUMENT INDEX -->
<div class="section">
    <h2>11. Document Index</h2>
    ${documents.length === 0
        ? '<p style="color:#64748B; font-size:10px;">No documents uploaded.</p>'
        : `
    <table>
        <tr>
            <th>Document Type</th>
            <th>File Name</th>
            <th>Uploaded By</th>
            <th>Upload Date</th>
            <th>Expiry</th>
            <th>Verified</th>
        </tr>
        ${documents.map(doc => `
            <tr>
                <td>${formatDocType(doc.docType)}</td>
                <td>${doc.fileName}</td>
                <td>${doc.uploadedBy?.firstName} ${doc.uploadedBy?.lastName}</td>
                <td>${formatDate(doc.createdAt)}</td>
                <td>${doc.expiryDate ? formatDate(doc.expiryDate) : '-'}</td>
                <td>${doc.verifiedAt
                    ? '<span class="badge badge-green">VERIFIED</span>'
                    : '<span class="badge badge-grey">UNVERIFIED</span>'}</td>
            </tr>
        `)}
    </table>
    `}
    </div>

```

```
    `).join('')}  
</table>  
  `}  
</div>
```

```
<!-- DISCLAIMER -->
```

```
<div class="disclaimer">
```

```
  <strong>Important Notice:</strong> This evidence bundle was  
  generated by Proplet property management software on  
  ${formatDate(generatedAt)} by ${generatedBy}.
```

```
  Proplet is a record keeping tool. This document does not  
  constitute legal advice and does not guarantee compliance  
  with any legislation. Always consult a qualified solicitor  
  for specific legal guidance. The accuracy of this document  
  depends on the completeness and accuracy of data entered  
  into Proplet by the agency.
```

```
</div>
```

```
</body>
```

```
</html>`;
```

```
}
```

```
// — Helper functions
```

```
function formatDate(date) {
```

```
  if (!date) return '-';
```

```
  return new Date(date).toLocaleDateString('en-GB', {  
    day: '2-digit', month: 'short', year: 'numeric'  
  });
```

```
}
```

```
function formatPeriod(date) {
```

```
  return new Date(date).toLocaleDateString('en-GB', {  
    month: 'long', year: 'numeric'  
  });
```

```
}
```

```
function formatAddress(address) {
```

```
  if (!address) return 'Not recorded';
```

```
  return [address.line1, address.line2, address.city,  
    address.postcode].filter(Boolean).join(', ');
```

```
}
```

```
function formatPropertyType(type) {
```

```
  const map = {
```

```
    terraced: 'Terraced House', semi_detached: 'Semi-Detached',  
    detached: 'Detached', flat: 'Flat', bungalow: 'Bungalow',
```

```

    maisonette: 'Maisonette', studio: 'Studio', hmo: 'HMO',
    other: 'Other'
  };
  return map[type] || type;
}

function formatPaymentMethod(method) {
  const map = {
    standing_order: 'Standing Order', bacs_transfer: 'BACS Transfer',
    direct_debit: 'Direct Debit', cash: 'Cash',
    cheque: 'Cheque', other: 'Other'
  };
  return map[method] || method;
}

function formatRTRDocType(type) {
  const map = {
    uk_passport: 'UK Passport',
    eu_settled_status: 'EU Settled Status',
    eu_pre_settled_status: 'EU Pre-Settled Status',
    biometric_residence_permit: 'Biometric Residence Permit',
    visa: 'Visa', certificate_of_naturalisation: 'Certificate of Naturalisation',
    other_uk_right: 'Other UK Right to Rent'
  };
  return map[type] || type || 'Not recorded';
}

function formatDocType(type) {
  const map = {
    tenancy_agreement: 'Tenancy Agreement',
    how_to_rent: 'How to Rent Guide',
    prescribed_info: 'Prescribed Information',
    deposit_certificate: 'Deposit Certificate',
    section8_notice: 'Section 8 Notice',
    section13_notice: 'Section 13 Notice',
    checkout_report: 'Checkout Report',
    inventory: 'Inventory', other: 'Other'
  };
  return map[type] || type;
}

function getAmlBadgeClass(result) {
  const map = { pass: 'badge-green', refer: 'badge-amber',
    fail: 'badge-red' };
  return map[result] || 'badge-grey';
}

```

```

function getRTRLLabel(tenant) {
  const checked = tenant.tenantData?.rightToRentChecked;
  if (!checked) return 'NOT CHECKED';
  const expiry = tenant.tenantData?.rightToRentExpiry;
  if (!expiry) return 'VALID';
  const days = Math.floor((new Date(expiry) - new Date()) / 86400000);
  if (days < 0) return 'EXPIRED';
  if (days <= 28) return `EXPIRES IN ${days} DAYS`;
  return 'VALID';
}

function getRTRBadgeClass(tenant) {
  const label = getRTRLLabel(tenant);
  if (label === 'VALID') return 'badge-green';
  if (label === 'NOT CHECKED' || label === 'EXPIRED') return 'badge-red';
  return 'badge-amber';
}

function getCertBadge(cert) {
  if (!cert) return '<span class="badge badge-grey">NOT UPLOADED</span>';
  const days = Math.floor(
    (new Date(cert.expiryDate) - new Date()) / 86400000
  );
  if (days < 0) return '<span class="badge badge-red">EXPIRED</span>';
  if (days <= 30) return `<span class="badge badge-amber">EXPIRES IN ${days}
  DAYS</span>`;
  return '<span class="badge badge-green">VALID</span>';
}

function getPaymentBadgeClass(status) {
  const map = { paid: 'badge-green', partial: 'badge-amber',
    overdue: 'badge-red', due: 'badge-amber',
    pending: 'badge-grey', waived: 'badge-grey' };
  return map[status] || 'badge-grey';
}

function getNoticeBadgeClass(outcome) {
  const map = { pending: 'amber', withdrawn: 'grey',
    complied: 'green', court_filed: 'red',
    accepted: 'green', challenged: 'red' };
  return map[outcome] || 'grey';
}

function getDepositStatus(tenancy) {
  if (!tenancy.deposit.amount) return 'No deposit taken';
  const deadline = new Date(tenancy.startDate);
  deadline.setDate(deadline.getDate() + 30);

```

```

    return `

```

```

=====
SECTION 7 - UI BUTTON AND DOWNLOAD FLOW
=====

```

```

// components/EvidenceVaultButton.tsx

import { useFetcher } from 'react-router';

export function EvidenceVaultButton({ tenancyId }) {
  const fetcher = useFetcher();
  const isGenerating = fetcher.state !== 'idle';

  // When generation completes and we have a download URL
  useEffect(() => {
    if (fetcher.data?.downloadUrl) {
      // Trigger browser download automatically
      const link = document.createElement('a');
      link.href = fetcher.data.downloadUrl;
      link.download = `evidence-bundle-${tenancyId}.pdf`;
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
    }
  }, [fetcher.data]);

  return (
    <div>
      <fetcher.Form
        method="POST"
        action={`\api/evidence-vault/${tenancyId}/generate`}
      >

```

```

    <button
      type="submit"
      disabled={isGenerating}
      className="..."
    >
      {isGenerating ? (
        <>
          <Spinner />
          Generating bundle...
        </>
      ) : (
        <>
          <DocumentIcon />
          Generate Evidence Bundle
        </>
      )}
    </button>
  </fetcher.Form>

  {fetcher.data?.error && (
    <p className="text-red-600 text-sm mt-2">
      {fetcher.data.error}
    </p>
  )}

  {fetcher.data?.downloadUrl && (
    <p className="text-green-600 text-sm mt-2">
      Bundle generated. Download starting...
    </p>
  )}
</div>
);
}

```

WHERE IT APPEARS ON THE TENANCY DETAIL PAGE:

- Top right of the page header (primary location)
- Next to the Edit button
- Also accessible from a generate button in the compliance summary strip area

SECTION 8 - PUPPETEER SETUP ON VERCEL

Puppeteer on Vercel requires specific configuration.
 Standard Puppeteer does not work in serverless out of the box.

USE: @sparticuz/chromium + puppeteer-core

This is the correct package combination for serverless:

```
npm install @sparticuz/chromium puppeteer-core
```

```
// In your PDF generation function:
import chromium from '@sparticuz/chromium';
import puppeteer from 'puppeteer-core';

export async function generatePDF(html) {
  const browser = await puppeteer.launch({
    args: chromium.args,
    defaultViewport: chromium.defaultViewport,
    executablePath: await chromium.executablePath(),
    headless: chromium.headless,
  });

  const page = await browser.newPage();
  await page.setContent(html, { waitUntil: 'networkidle0' });

  const pdfBuffer = await page.pdf({
    format: 'A4',
    printBackground: true,
    margin: { top: '20mm', right: '15mm',
      bottom: '20mm', left: '15mm' },
    displayHeaderFooter: true,
    headerTemplate: `...`,
    footerTemplate: `...`,
  });

  await browser.close();
  return pdfBuffer;
}
```

VERCEL FUNCTION SIZE LIMIT:

The @sparticuz/chromium package is large (~40MB compressed).
Vercel hobby plan has a 50MB function size limit.
Vercel Pro plan has a 250MB limit.
If you hit size limits: move generation to a separate
Express server on Railway or Render (~£5/month).

ALTERNATIVE IF VERCEL LIMITS ARE A PROBLEM:

Use html-pdf-node package instead of Puppeteer.
Smaller package, simpler setup, slightly less control.
Good fallback option if Puppeteer proves problematic.

=====

SECTION 9 - S3 STORAGE FOR BUNDLES

=====

S3 key structure for generated bundles:

agency/{agencyId}/evidence-bundles/{tenancyId}/bundle.pdf

IMPORTANT:

Each generation overwrites the previous bundle.

No versioning -- always the freshest data.

File is not permanent -- regenerated on demand.

Presigned URL expires after 1 hour.

After 1 hour agent must generate again to download.

OPTIONAL: add a 7-day S3 lifecycle rule to auto-delete old bundle files to save storage costs.

The bundle can always be regenerated from live data.

S3 BUCKET CONFIG:

Same bucket as other documents.

evidence-bundles/ prefix keeps them organised.

All public access blocked (same as all other files).

=====

SECTION 10 - BUILD ORDER

=====

STEP 1 - Install Puppeteer dependencies (30 mins)

npm install @sparticuz/chromium puppeteer-core

Test locally: generate a simple PDF from HTML string.

Verify: PDF opens and renders correctly.

Done when: basic PDF generation works locally.

STEP 2 - Build fetchBundleData function (1-2 hrs)

lib/evidenceVault.server.ts

All MongoDB queries from Section 3.

Test: call with a real tenancyId, verify all data returned.

Check: all populate fields return correctly.

Done when: complete data object returned without errors.

STEP 3 - Build HTML template (3-4 hrs)

lib/bundleTemplate.server.ts

All 11 sections from Section 6.

All helper functions.

Test: render HTML with mock data, open in browser.
Check: layout looks professional, no broken sections.
Check: badges render correctly for each status.
Check: tables format correctly with data.
Done when: HTML renders correctly in Chrome browser.

STEP 4 - Build generatePDF function (1 hr)

lib/evidenceVault.server.ts
Puppeteer setup from Section 8.
Header and footer templates.
Test: generate PDF from rendered HTML.
Check: PDF opens, fonts load, layout matches browser view.
Check: page numbers appear in footer.
Check: header appears on each page.
Done when: PDF generates and looks professional.

STEP 5 - Build server action (1 hr)

routes/api.evidence-vault.\$tenancyId.generate.tsx
Auth check, agency isolation, data fetch, generate, upload.
Test: POST to route, verify PDF uploaded to S3.
Test: verify presigned URL returned and works.
Test: verify Note created in audit trail.
Done when: end-to-end generation works via API.

STEP 6 - Build EvidenceVaultButton component (1 hr)

components/EvidenceVaultButton.tsx
useFetcher form submission.
Loading state with spinner.
Auto-download trigger on success.
Error display on failure.
Done when: button generates and auto-downloads PDF.

STEP 7 - Add button to tenancy detail page (30 mins)

Import EvidenceVaultButton into tenancy detail.
Add to page header top right.
Test: click button, verify loading state shows.
Test: verify PDF downloads automatically.
Done when: full flow works from tenancy detail page.

STEP 8 - Integration tests (1 hr)

TEST 1: Standard APT tenancy
Create tenancy with all fields populated.
Upload gas cert, EICR, EPC.
Record 3 months of payments.
Record How to Rent as served.
Record deposit protection.

Click Generate Evidence Bundle.
Verify: PDF downloads automatically.
Verify: all 11 sections present in PDF.
Verify: dates formatted correctly (UK format).
Verify: status badges correct colours.
Verify: audit note created in tenancy notes.

TEST 2: Tenancy with arrears

Create tenancy with 2 overdue payments.
Generate bundle.
Verify: arrears warning section appears in PDF.
Verify: Ground 8 eligibility mentioned.

TEST 3: Tenancy with Section 8 notice

Record a Section 8 notice.
Generate bundle.
Verify: Section 8 notices section appears.
Verify: grounds listed correctly.

TEST 4: Tenancy with missing data

Create minimal tenancy (no certs, no payments).
Generate bundle.
Verify: missing items show correct 'not uploaded' status.
Verify: PDF still generates without errors.

TEST 5: Agency isolation

Log in as agency 2.
Try to generate bundle for agency 1 tenancy.
Verify: 404 returned, no PDF generated.

TEST 6: Large tenancy (performance)

Tenancy with 24 months of payments.
Multiple Section 8 notices.
10+ documents.
Verify: PDF generates within 10 seconds.
Verify: all data present and correctly formatted.

TOTAL ESTIMATED TIME: 10-14 hours

SECTION 11 - RULES FOR CODING AGENT

1. PUPPETEER ON VERCEL

Use @sparticuz/chromium + puppeteer-core.
NOT standard puppeteer package.

Standard puppeteer will fail in serverless environment.

2. INLINE CSS ONLY IN TEMPLATE

No external stylesheets in the HTML template.

Puppeteer in serverless cannot load external CSS.

All styles must be inline or in a <style> tag
in the same HTML document.

3. NO EXTERNAL IMAGES

Do not reference external image URLs in the template.

Use text and CSS styling only.

If you want a logo: embed as base64 inline.

4. AGENCY ISOLATION CRITICAL

Verify tenancy belongs to user.agencyId before
generating any bundle.

A bundle for the wrong agency would expose
another agency's data -- serious GDPR violation.

5. S3 KEY OVERWRITES PREVIOUS BUNDLE

Same S3 key used each time for the same tenancy.

This is intentional -- always freshest data.

No versioning needed for evidence bundles.

6. DISCLAIMER MUST APPEAR ON EVERY BUNDLE

The legal disclaimer in Section 6 must appear
at the bottom of every generated PDF.

Never remove or shorten it.

7. AUDIT NOTE ON GENERATION

Every bundle generation creates a Note:

'Evidence bundle generated by [name]'

This creates an immutable record of when bundles
were generated and by whom.

Important for legal defensibility.

8. PDF GENERATION IS SLOW

Puppeteer typically takes 3-8 seconds.

Show a clear loading state on the button.

Do not let the user think the page has frozen.

'Generating bundle...' with a spinner is sufficient.

9. HANDLE PUPPETEER FAILURES GRACEFULLY

Wrap generatePDF in try/catch.

Return a clear error message to the UI.

Log the full error server-side for debugging.

Never let a PDF failure cause an unhandled 500 error.

10. DATE FORMAT IS UK THROUGHOUT

All dates in the PDF use en-GB locale.

DD Mon YYYY format (e.g. 15 Mar 2026).

Never use US format (Mar 15, 2026) in the template.

=====
END OF EVIDENCE VAULT IMPLEMENTATION PLAN
=====